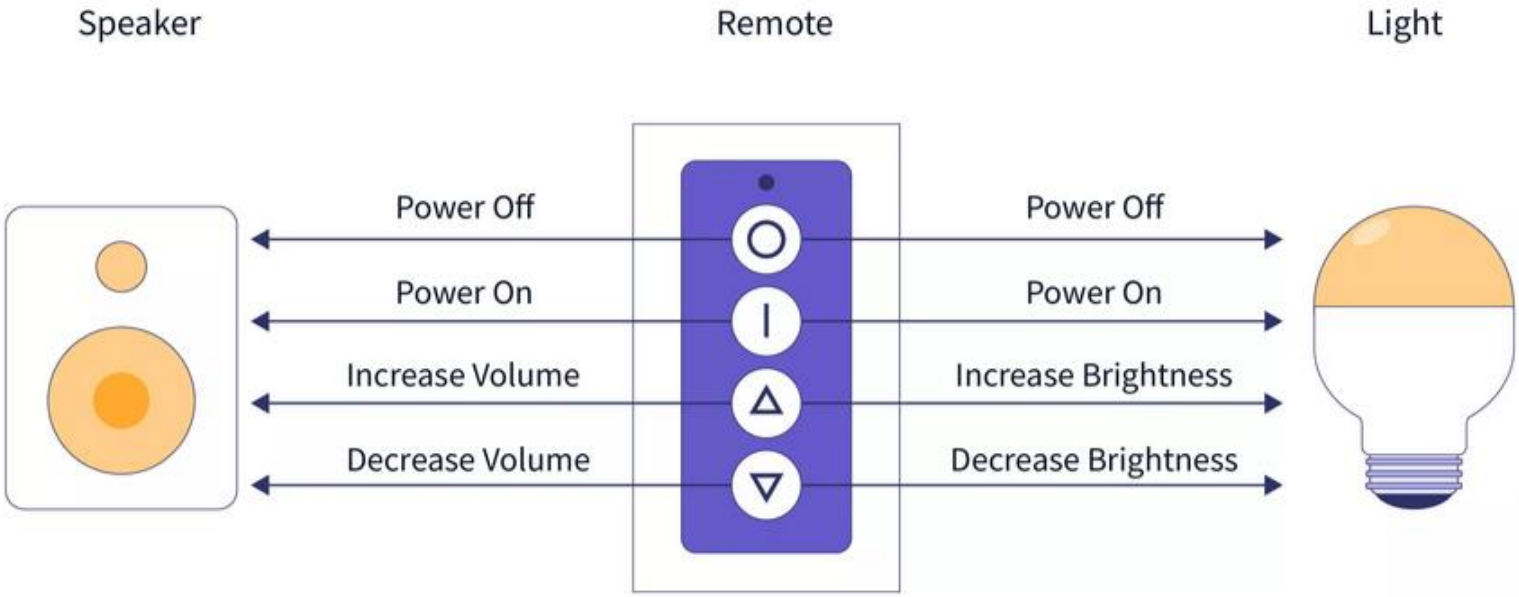
Поведенческие паттерны. Команда

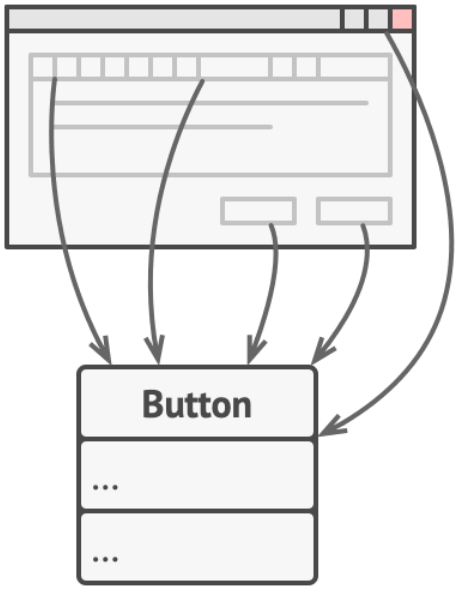


Паттерн Команда
Command (Действие, Транзакция, Action)

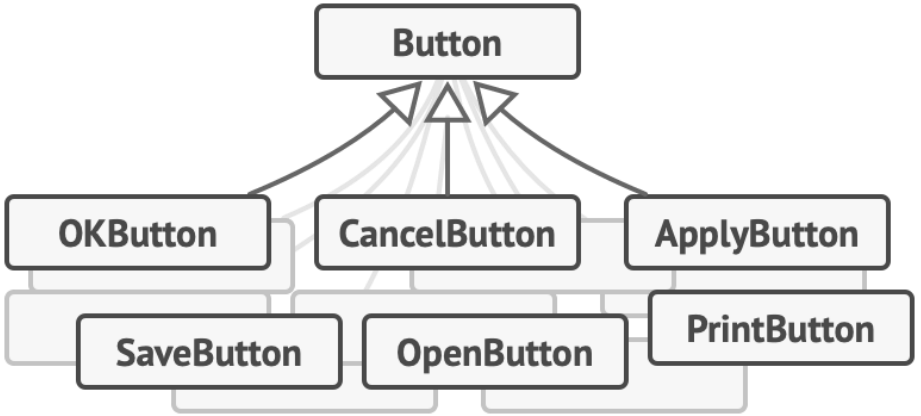


Паттерн **Command (Команда)** – позволяет представить запрос в виде объекта, позволяя клиенту конфигурировать запрос (задавая параметры для его обработки), ставить запросы в очередь, протоколировать запросы, а также поддерживать отмену операций.

Создание иерархии элементов управления



Все кнопки приложения унаследованы от одного класса, но реализуют разную функциональность



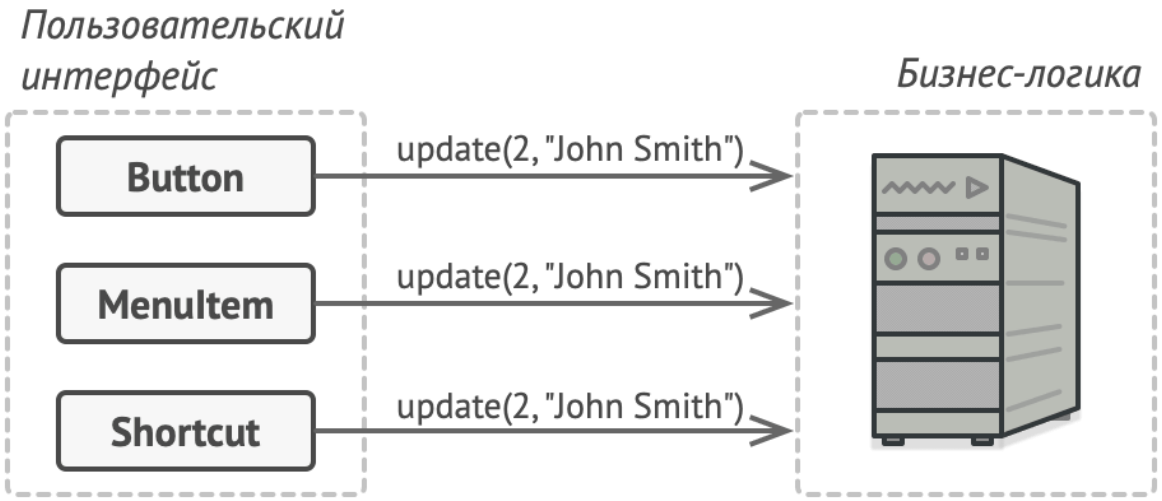


- Множество подклассов кнопок (рост количества классов; зависимость кода кнопок от бизнес-логики)

Несколько классов дублируют одну и ту же функциональность (одно и то же действие можно выполнить с помощью разных элементов интерфейса)

Решение

Структурирование программы на слои пользовательского интерфейса и бизнес-логики.



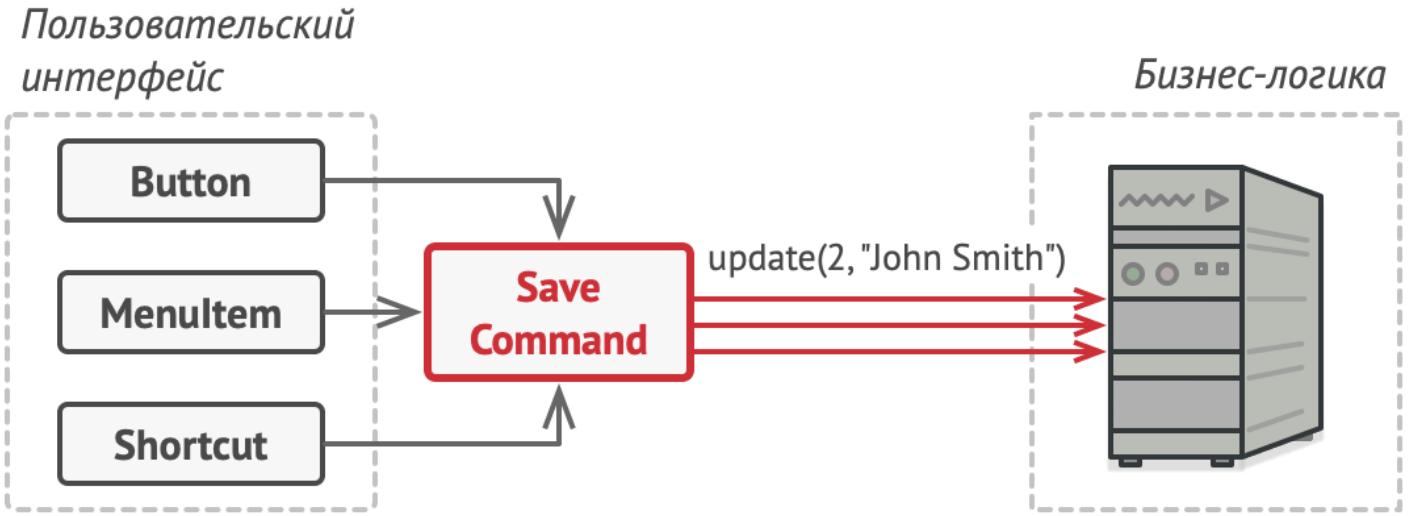
Прямой доступ из UI в бизнес-логику.

Паттерн **Команда** предлагает не отправлять такие вызовы напрямую.
Вместо этого, каждый вызов, отличающийся от других, следует «завернуть» в собственный класс с единственным методом, который и будет осуществлять вызов.
Такие объекты называют *командами*.

Решение

С объектом *интерфейса* связан объект *команды*, который знает, кому и в каком виде следует отправлять запросы.

Когда объект *интерфейса* будет готов передать *запрос*, он вызовет метод *команды*, который реализует действие.



Доступ из UI в бизнес-логику через команду

Классы команд можно объединить под общим интерфейсом с единственным *методом запуска*.

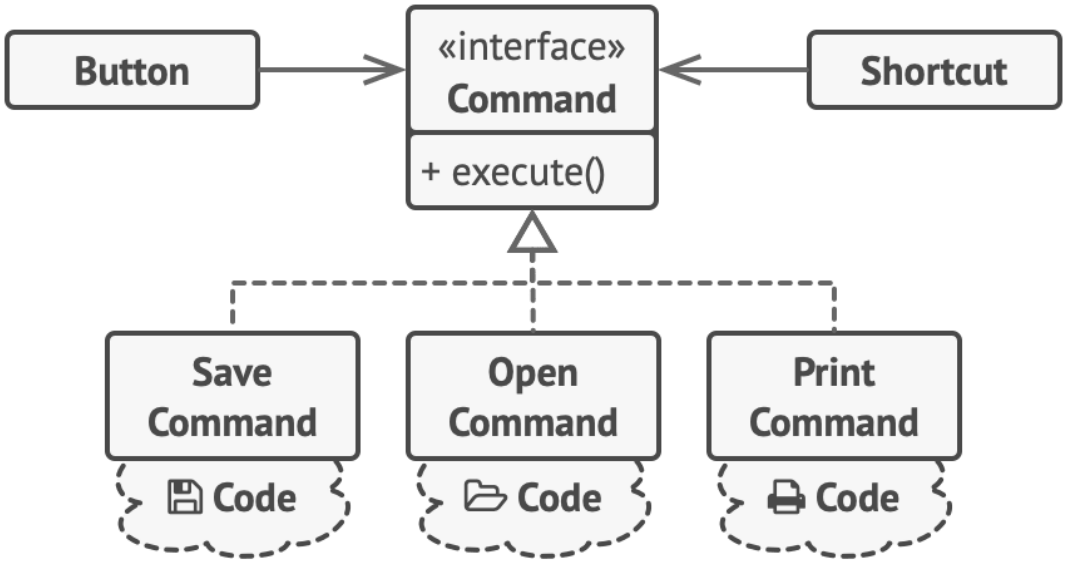
После этого одни и те же отправители смогут работать с различными командами, не привязываясь к их классам.

Команды можно будет взаимозаменять динамически, изменяя итоговое поведение отправителей.

Решение

Параметры, с которыми должен быть вызван метод объекта получателя, можно заранее сохранить в полях объекта-команды.

Благодаря этому, объекты, отправляющие запросы, не связаны с получателем. Вся эта информация скрыта внутри команды.

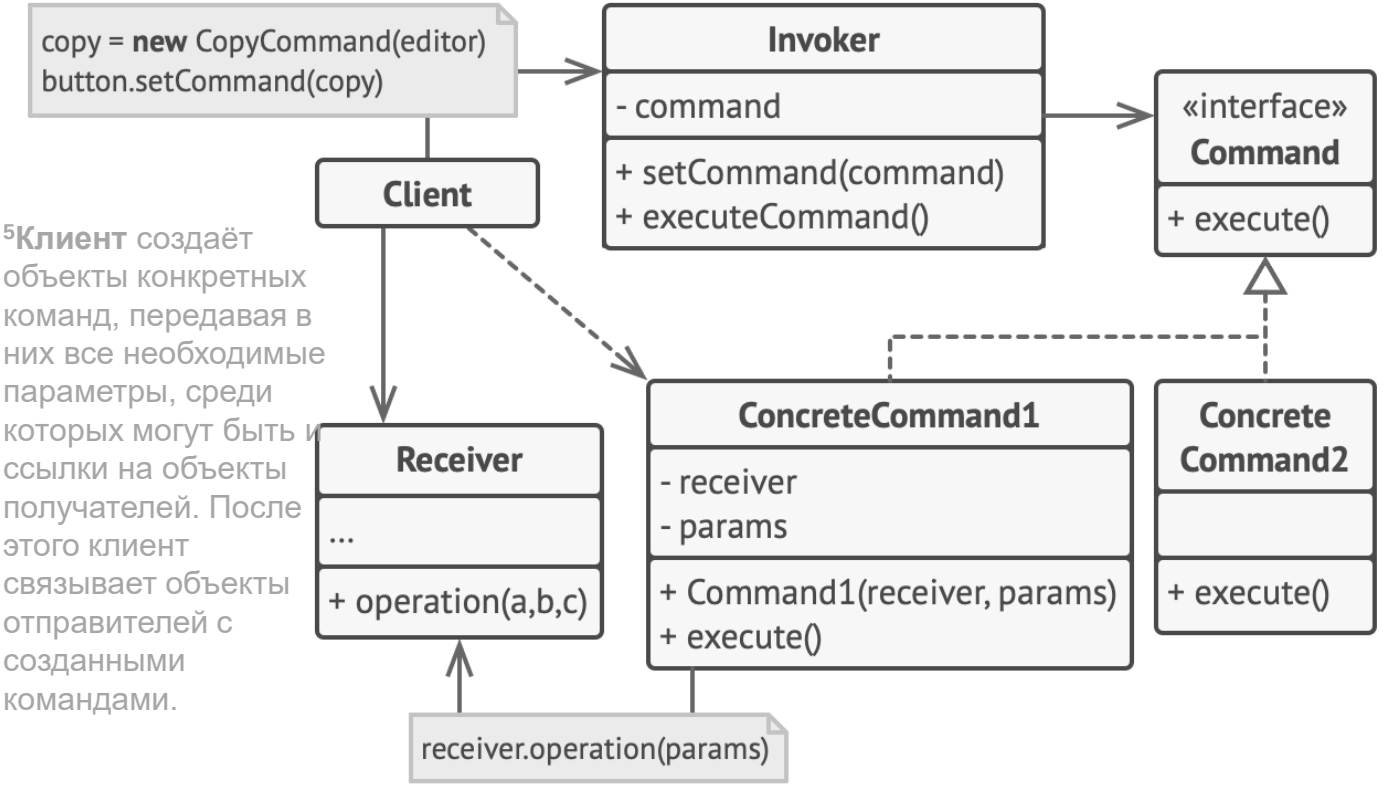


Классы UI делегируют работу командам.

Структура паттерна

[1]**Отправитель** хранит ссылку на объект команды и обращается к нему, когда нужно выполнить какое-то действие. Отправитель работает с командами только через их общий интерфейс. Он не знает, какую конкретно команду использует, так как получает готовый объект команды от клиента.

[2]**Команда** описывает общий для всех конкретных команд интерфейс. Обычно здесь описан всего один метод для запуска команды.

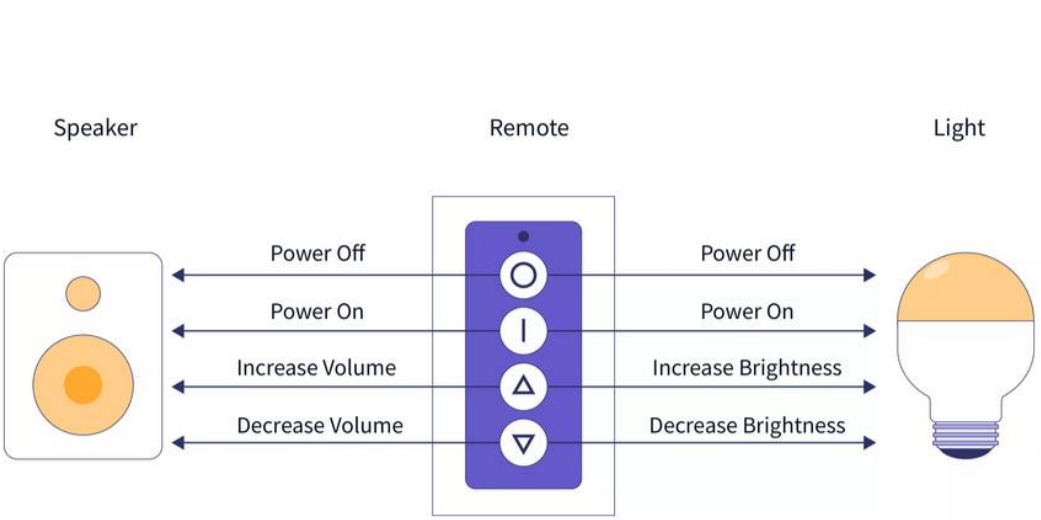


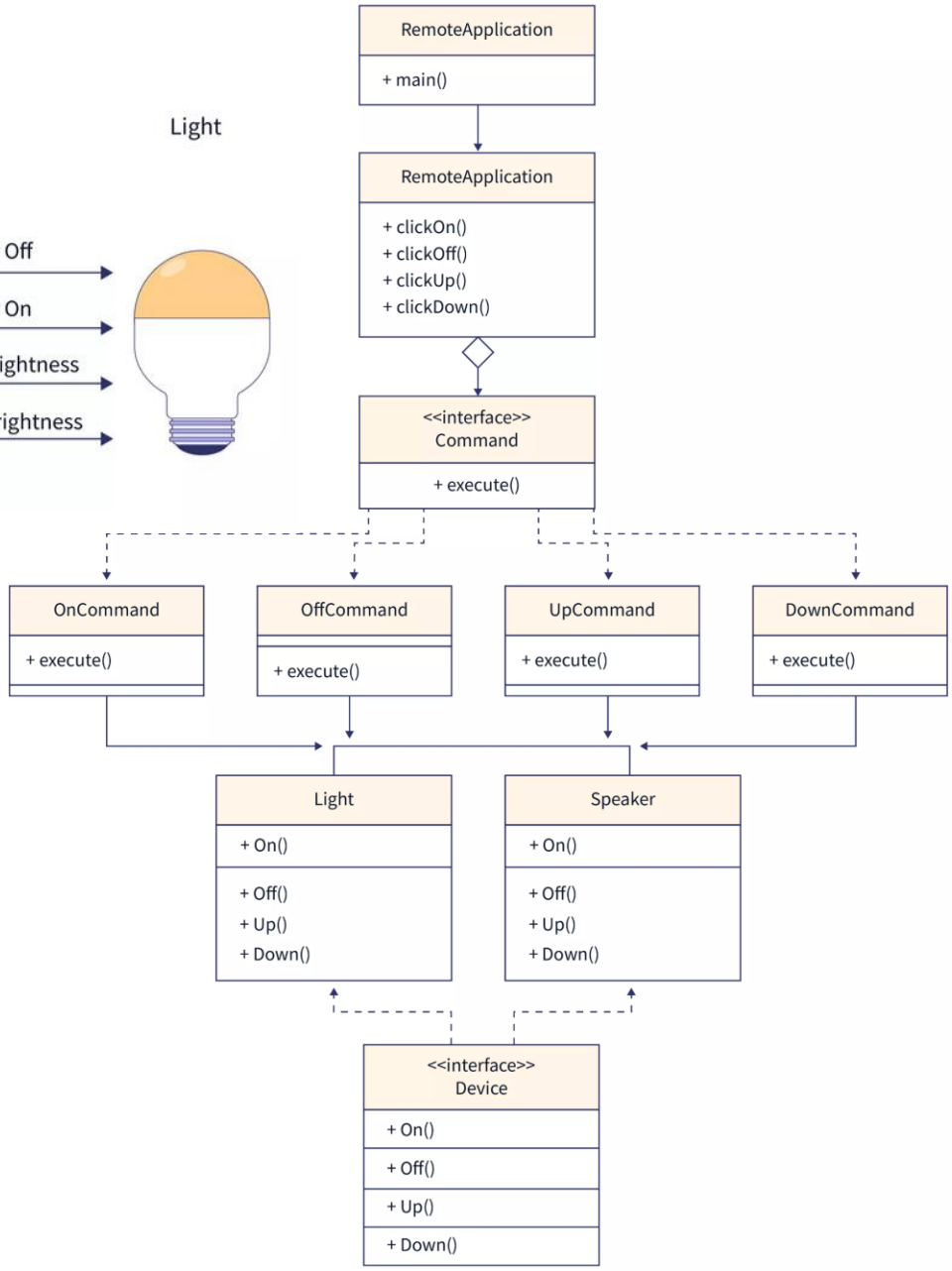
[3]**Конкретные команды** реализуют различные запросы, следуя общему интерфейсу команд. Обычно команда не делает всю работу самостоятельно, а лишь передаёт вызов получателю, которым является один из объектов бизнес-логики.
Параметры, с которыми команда обращается к получателю, следует хранить в виде полей. В большинстве случаев объекты команд можно сделать неизменяемыми, передавая в них все необходимые параметры только через конструктор.

[5]**Клиент** создаёт объекты конкретных команд, передавая в них все необходимые параметры, среди которых могут быть и ссылки на объекты получателей. После этого клиент связывает объекты отправителей с созданными командами.

[4]**Получатель** содержит бизнес-логику программы. В этой роли может выступать практически любой объект. Обычно команды перенаправляют вызовы получателям. Но иногда, чтобы упростить программу, можно реализовать необходимые действия в классе команды.

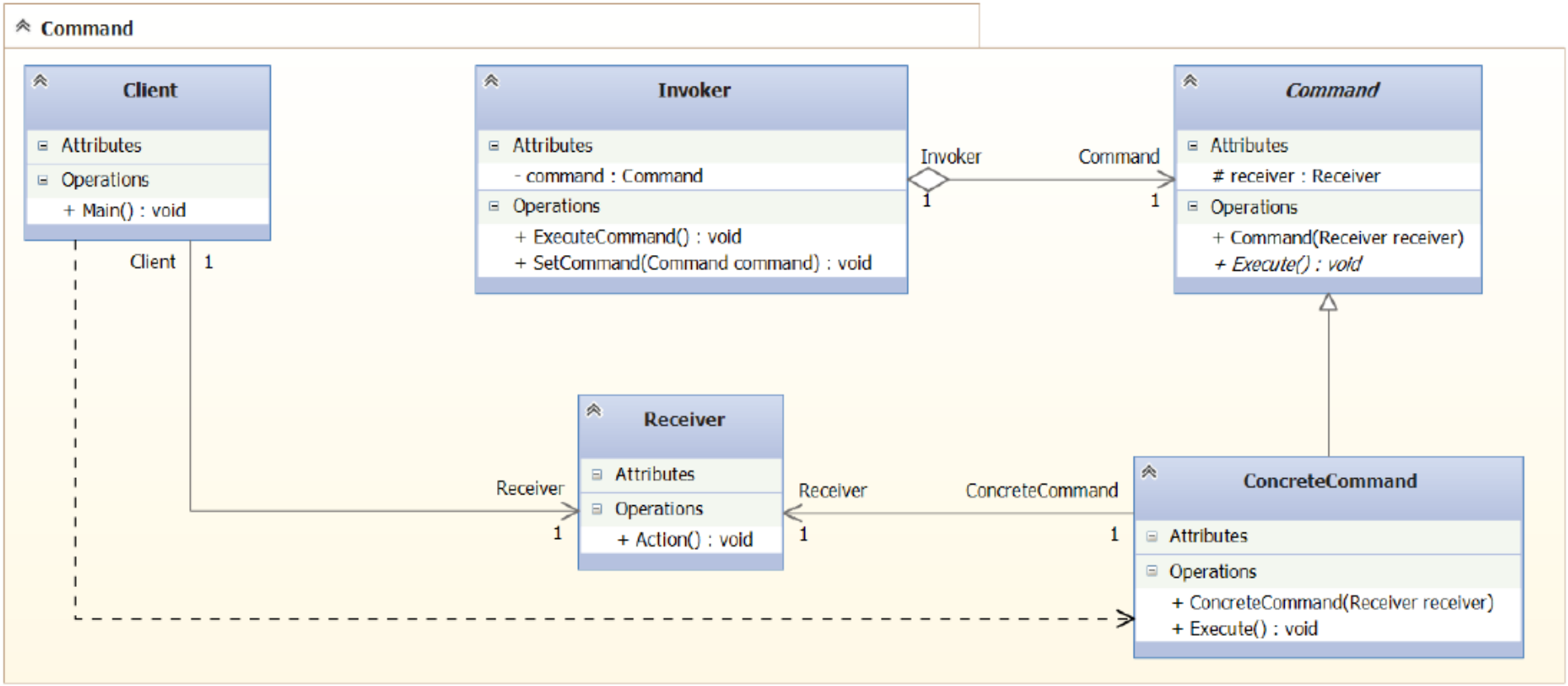
Пример





Реализация программного интерфейса универсального пульта ДУ

Структура паттерна UML



Структура паттерна на языке C#

```
class Program
{
    static void Main()
    {
        Receiver receiver = new Receiver();
        Command command = new ConcreteCommand(receiver);
        Invoker invoker = new Invoker();

        invoker.StoreCommand(command);
        invoker.ExecuteCommand();
    }
}
```

`Client` - **Клиент**:
Создает объект класса `ConcreteCommand` и устанавливает его получателя - объект класса `Receiver`.

`Invoker` - **Инициатор**:
Обращается к объекту-команде для выполнения запроса.

`Receiver` - **Получатель**:
Обладает функциональностью достаточной для выполнения определенного запроса. В роли получателя запроса может выступать любой класс.

```
class Invoker
{
    Command command;

    public void StoreCommand(Command command)
    {
        this.command = command;
    }

    public void ExecuteCommand()
    {
        command.Execute();
    }
}
```

```
abstract class Command
{
    protected Receiver receiver;

    public Command(Receiver receiver)
    {
        this.receiver = receiver;
    }

    public abstract void Execute();
}
```

```
class Receiver
{
    public void Action()
    {
        Console.WriteLine("Receiver");
    }
}
```

```
class ConcreteCommand : Command
{
    public ConcreteCommand(Receiver receiver)
        : base(receiver)
    {
    }

    public override void Execute()
    {
        receiver.Action();
    }
}
```

`Command` - **Команда**:
Предоставляет интерфейс для выполнения операции `ConcreteCommand` - **Конкретная команда**:
Представляет собой объектно-ориентированное выражение соответствия между объектом класса `Receiver` и выполняемым им действием `Action`.
Реализует операцию `Execute` которая вызывает соответствующие операции объекта класса `Receiver`.

Отношения между участниками

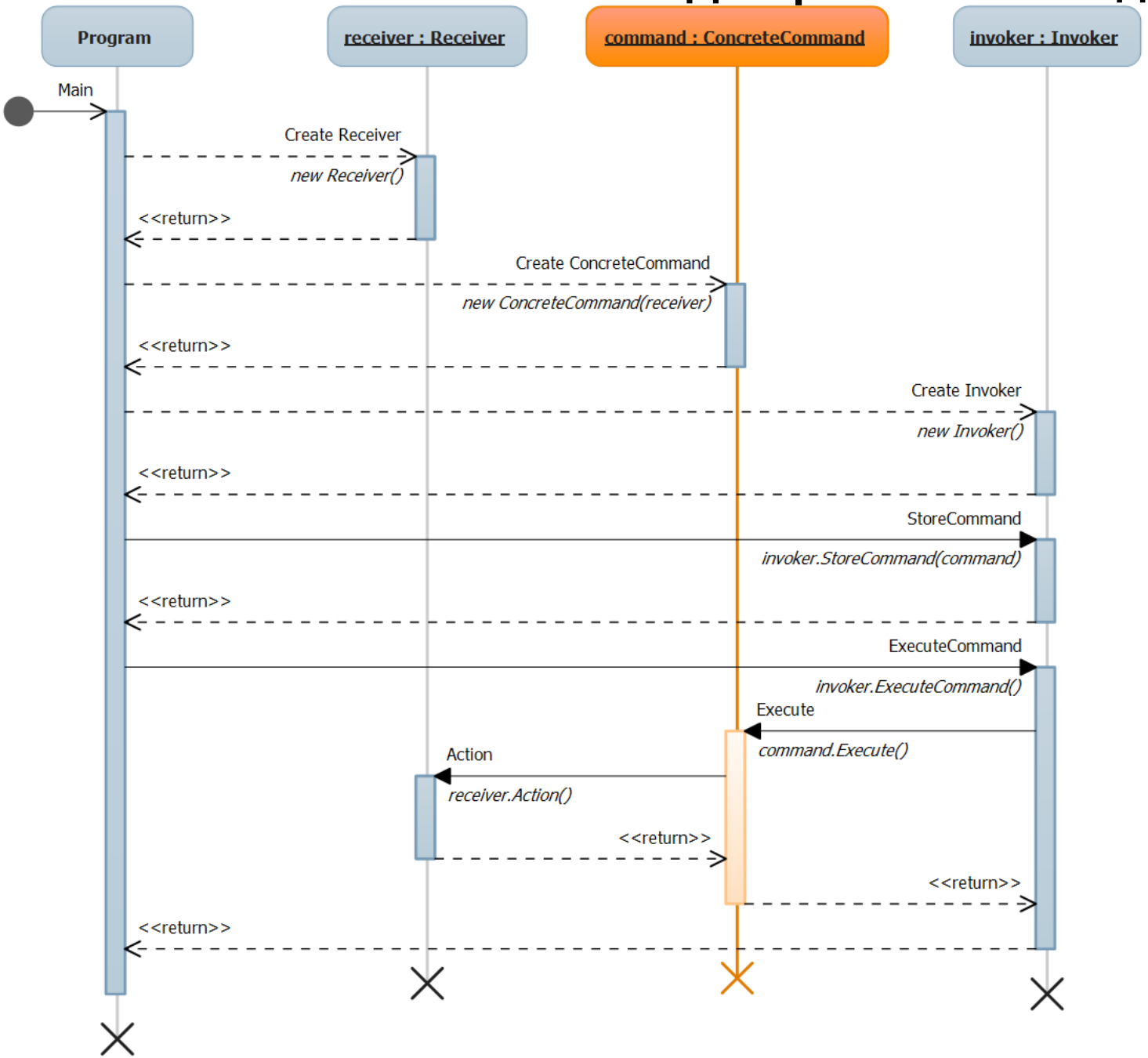
Отношения между классами

- Класс Invoker связан связью отношения агрегации с абстрактным классом Command.
- Класс ConcreteCommand связан связью отношения наследования с абстрактным классом Command и связью отношения ассоциации с конкретным классом Receiver.

Отношения между объектами

- Клиент создает объект класса ConcreteCommand и устанавливает для него получателя (Receiver).
- Объект класса Invoker сохраняет в списке историй команд объект класса ConcreteCommand и отправляет запрос на выполнение команды вызывая метод Execute на объекте-команде. Если требуется поддержка отмены (Undo) и повтора (Redo) операций, то объект ConcreteCommand перед выполнением тела метода Execute должен сохранить информацию о своем текущем состоянии, достаточную для выполнения отката (Undo).
- Объект класса ConcreteCommand вызывает операции (Action) объекта-получателя (Receiver) для выполнения запроса.

Диаграмма взаимодействия



Применение

Когда необходимо параметризовать объекты выполняемым действием

Команда превращает операции в объекты. А объекты можно передавать, хранить и взаимозаменять внутри других объектов.

Когда необходимо ставить операции в очередь, выполнять их по расписанию или передавать по сети.

Как и любые другие объекты, команды можно сериализовать, то есть превратить в строку, чтобы потом сохранить в файл или базу данных. При необходимости её можно извлечь снова превратить в объект команды и выполнить. Таким же образом команды можно передавать по сети, логировать или выполнять на удалённом сервере.

Когда нужно реализовать операцию отмены команды.

История команд выглядит как *стек*, в который попадают все выполненные объекты команд. Каждая команда перед выполнением операции сохраняет текущее состояние объекта, с которым она будет работать. После выполнения операции копия команды попадает в стек истории, все ещё неся в себе сохранённое состояние объекта. Если потребуется отмена, программа возьмёт последнюю команду из истории и возобновит сохранённое в ней состояние.

Шаги реализации

1.Создать общий интерфейс команд и определить в нём метод запуска.

2.Создать классы конкретных команд. В каждом классе должно быть поле для хранения ссылки на один или несколько объектов-получателей, которым команда будет перенаправлять основную работу.

- Команда должна иметь поля для хранения параметров, которые нужны при вызове методов получателя. Значения всех этих полей команда должна получать через конструктор.
- Реализовать основной метод команды, вызывая в нём те или иные методы получателя.

3. Добавить в классы отправителей поля для хранения команд. Обычно объекты-отправители принимают готовые объекты команд извне — через конструктор либо через сеттер поля команды.

4.Изменить основной код отправителей так, чтобы они делегировали выполнение действия команде.

5.Порядок инициализации объектов должен выглядеть так:

1. Создание объектов *получателей*.
2. Создание объектов *команд*, связывание их с *получателями*.
3. Создание объектов *отправителей*, связывание их с *командами*.

Преимущества и недостатки

+ Убирает прямую зависимость между объектами, вызывающими операции, и объектами, которые их непосредственно выполняют.

+ Позволяет реализовать простую отмену и повтор операций.

+ Позволяет реализовать отложенный запуск операций.

+ Позволяет собирать сложные команды из простых.

+ Реализует принцип *открытости/закрытости*.

- Усложняет код программы из-за введения множества дополнительных классов.